

# Taskdeck: Future Expansion & Improvements Checklist

## 1. Context and Existing Roadmap

Taskdeck's core is a **local-first kanban platform** with a .NET 8 backend and Vue 3 frontend. The current roadmaps (Horizon A/B/C) and backlog already plan the following key work:

- **Security & Identity:** Claims-first identity retrofit across all controllers; adoption of a strict 401/403/404 policy; expansion of unauthorized/forbidden/cross-user tests; standardization of error contracts <sup>1</sup> <sup>2</sup> .
- **Automation Hardening:** Integration of a production LLM provider; extended planner/executor semantics; guardrails for invalid proposals; chat-to-project bootstrap <sup>3</sup> <sup>4</sup> .
- **UX & Operator Enhancements:** keyboard navigation and command palette improvements; archive restore coherence; drag/edit guardrails; activity discoverability; input-assist modules; improved logs and ops interfaces <sup>5</sup> .
- **Starter Packs & QA:** package manifest schema and apply endpoint; first-party starter packs and deterministic QA fixtures; UI for browsing packages <sup>6</sup> .
- **Tech Debt:** nullability warning reduction, log query performance, full DB export/import decision <sup>7</sup> .

The **testing harness** is unusually comprehensive for an early-stage project: 477 backend tests (domain/application/API/CLI/architecture), 245 frontend unit tests and 11 end-to-end Playwright tests, all enforced by CI across Windows and Linux <sup>8</sup> <sup>9</sup> . The current strategy emphasizes high contract coverage and a small E2E smoke suite. This level of rigor is more typical of mature teams than solo projects, providing confidence during refactors.

## 2. Additional Feature and Improvement Ideas

The following ideas build on the existing roadmap and align with Taskdeck's local-first, automation-driven ethos. They are grouped by theme to aid prioritization.

### 2.1 Real-Time Collaboration & Communication

1. **SignalR/WebSockets for Live Updates** – Implement real-time push updates so board changes propagate instantly to all connected clients. This improves collaboration efficiency and scales better than polling. Use sticky sessions or a Redis backplane when running multiple instances.
2. **Notifications Framework** – Provide in-app, email or system notifications for events (card assignments, due dates, automation results, proposal approvals). Offer per-board/user preferences and digest options.
3. **Collaborative Editing & Presence** – Show who is viewing or editing a card; lock or merge concurrent edits gracefully; highlight changes in real time.
4. **Commenting and Mentions** – Add threaded comments and `@mentions` on cards or boards; integrate with notifications so mentions trigger alerts.

## 2.2 Multi-Tenancy & Data Isolation

1. **Tenant-Aware Architecture** – Refactor data models to include a `TenantId` and ensure all queries are scoped. Provide an admin panel to create/manage tenants.
2. **Tenant-Level RBAC** – Offer per-tenant roles (owner, admin, contributor, viewer) and board-level permissions (read-only, comment, edit). Integrate these with the existing claims-based identity model.
3. **Self-Hosted Instances** – Document and support multi-tenant or single-tenant deployments (Docker Compose or Helm charts), with per-tenant configuration (logo, domain, email templates).

## 2.3 Integrations & Extensibility

1. **Import/Export from Other Tools** – Provide import wizards for CSV, Trello, Jira or GitHub issues; export boards as JSON, CSV or Markdown. This complements the planned database export/import feature and lowers migration barriers.
2. **Third-Party Integrations** – Offer connectors for Slack/Teams (send notifications, create cards from messages), GitHub/GitLab (sync issues, link commits), and calendar services (convert cards with due dates into calendar events).
3. **Webhook & API** – Expose a public REST/GraphQL API and outbound webhooks so external systems can create/update boards and receive events. Document authentication (OAuth/API keys) and rate limits.
4. **Plugin System** – Architect a plugin/extension framework so advanced users can write custom automation modules, custom card fields, or integration adapters.

## 2.4 Advanced Automation & AI

1. **AI Summaries & Insights** – Use LLMs to summarise board status, surface overdue tasks, and suggest priorities or risk areas. Provide daily/weekly summary emails.
2. **Natural-Language Commands** – Allow users to create cards or move tasks via chat commands (e.g. “Add ‘Fix login bug’ to Project X column ‘To Do’”). Map parsed intents to actions.
3. **Recommendation Engine** – Suggest labels, due dates, or assignees based on historical patterns. Use reinforcement learning or simpler heuristics.
4. **Sentiment & Mood Detection** – Analyse comments and automation proposals to detect frustration or confusion; surface this to managers for intervention.
5. **AI-Powered Templates** – Generate new board templates or starter packs automatically from past successful boards or from natural language descriptions (“create a hiring pipeline board”).

## 2.5 Analytics & Reporting

1. **Board Metrics Dashboard** – Visualize throughput, cycle time, WIP counts, burndown/burnup charts, blocked tasks and user workloads. Provide filters by date range, label, member.
2. **Exportable Reports** – Generate PDF/CSV reports summarizing board activity, time-to-completion metrics, automation success rates, etc.
3. **Forecasting & Capacity Planning** – Use historical data to forecast completion dates and needed resources; suggest WIP limits or team size adjustments.
4. **Custom KPIs** – Let users define metrics and dashboards (e.g. number of tasks completed per sprint, average lead time).
5. **Audit & History Views** – Provide full audit history of changes (cards created, moved, edited, archived) with diff view and ability to revert changes.

## 2.6 User Experience & Accessibility

1. **Mobile & Desktop Apps** – Package Taskdeck as a Progressive Web App (PWA) or Electron app with offline sync for Android/iOS/desktop.
2. **Theme & Branding Customization** – Support dark/light themes and allow per-tenant color schemes; permit background images or board covers.
3. **Kanban Variants** – Add swimlanes, WIP limits per column or lane, swimlane grouping by assignee/priority, and quick filters for labels/dates.
4. **Calendar & Timeline Views** – Provide calendar and Gantt/timeline visualizations of tasks to plan work beyond the kanban board.
5. **Accessibility (a11y)** – Finish keyboard navigation, add screen-reader support (ARIA roles, alt text), ensure color contrast and text resizing, implement high-contrast mode.
6. **Inline Editing & Rich Text** – Support inline editing of card descriptions with rich text (markdown), images, attachments; drag-drop attachments to cards.
7. **Global Search & Quick Actions** – Implement fuzzy search across boards/cards and global hotkeys to create or jump to boards quickly.
8. **User Onboarding & Tour** – Provide an interactive onboarding tour for new users; include context-sensitive help and keyboard shortcut guides.

## 2.7 Data Privacy, Compliance & Security

1. **Encryption at Rest & in Transit** – Encrypt sensitive fields in the database; adopt transparent disk encryption for local DBs. Enforce TLS in production.
2. **GDPR & Data Portability** – Allow users to export all their personal data (boards, cards, comments) and support right-to-be-forgotten operations. Document data retention policies.
3. **Single Sign-On & MFA** – Integrate with OAuth/OIDC providers (Google, Microsoft, GitHub) and support optional MFA for accounts.
4. **Fine-Grained Permissions** – Extend RBAC to field-level or action-level permissions (e.g. only board admins can edit automations). Provide sharing links with expiration and view/comment only rights.
5. **Security Incident Response Plan** – Define procedures for vulnerability disclosure, incident handling, and periodic security audits (pen tests).
6. **Regular Dependency Audits** – Automate scanning for vulnerable packages (GitHub Dependabot, npm/yarn audit) and update processes.

## 2.8 Observability & Operations

1. **Structured Logging & Correlation** – Expand Serilog/OpenTelemetry usage to include correlation IDs across microservices; log key events with context but exclude secrets.
2. **Metrics & Monitoring** – Expose Prometheus metrics for request rates, latencies, error rates, queue lengths and automation performance; integrate with Grafana dashboards.
3. **Tracing** – Instrument the application with distributed tracing to follow requests through API, automation and DB layers.
4. **Health Checks & Self-Healing** – Implement readiness/liveness endpoints; integrate with orchestrators (Kubernetes/Container Apps) for automated restarts.
5. **Performance Benchmarks** – Establish baseline performance metrics (response times, throughput) and add automated benchmarks to detect regressions.
6. **Automated Backups & Disaster Recovery** – Regularly backup DB and attachments; provide restore procedures; support point-in-time recovery.
7. **Rollout Strategies** – Adopt blue/green or canary deployments for updates; automate rollback on failures.

## 2.9 Testing & Quality Engineering Enhancements

1. **Property-Based & Fuzz Testing** – Use property-based testing for data transformations (e.g. board merging, export/import) and fuzz testing for API inputs to uncover edge cases.
2. **Concurrency & Race Condition Tests** – Simulate multiple users moving cards concurrently and verify atomicity/consistency. Use integration tests or Playwright to orchestrate multiple sessions.
3. **Performance & Load Tests** – Add automated load testing (k6/JMeter) to simulate teams of 20+ users; monitor response times and resource usage.
4. **Cross-Browser & Mobile Testing** – Expand Playwright tests to cover Firefox and WebKit; test responsive layouts and mobile browsers.
5. **Snapshot & Visual Regression Tests** – Use tools like Playwright's snapshot or Storybook to detect visual regressions in UI components when styling changes.
6. **Mutation Testing** – Employ mutation testing (e.g. Stryker for .NET and JS) to measure test suite robustness and identify areas lacking assertions.
7. **Test Data Builders & Fixtures** – Refactor tests to use builders/factories for domain entities; share deterministic fixtures across unit and integration tests to reduce duplication.
8. **Test Coverage & Static Analysis** – Continue monitoring coverage; integrate code quality tools (StyleCop, ESLint) and security scanners (SonarQube, CodeQL).
9. **Ephemeral Databases for Integration Tests** – Use testcontainers or in-memory DBs to parallelize integration tests without cross-contamination.
10. **Release Candidate Gate** – Create a test suite that must pass before tagging a release (including manual test checklist, smoke E2E, performance benchmarks and security scans).

## 2.10 Documentation & Community

1. **Interactive Developer Portal** – Build a developer hub documenting API endpoints, webhook payloads, and integration examples; auto-generate from OpenAPI specifications.
2. **User Guides & Tutorials** – Write step-by-step guides for common workflows; provide video tutorials; maintain a FAQ section.
3. **Contribution Guidelines & Code of Conduct** – Encourage open-source contributions by documenting how to set up dev environments, run tests, and adhere to coding standards.
4. **Roadmap Transparency** – Publicly share the horizon plan and backlog; allow community feedback and voting on feature proposals.
5. **Localization** – Internationalize the UI and docs; allow users to contribute translations.

## 3. Notes on Testing Strategy Improvement

Taskdeck's current testing strategy is strong: unit, integration, API, CLI and architecture tests ensure domain and contract correctness, while a minimal E2E suite verifies critical paths <sup>10</sup>. To enhance it:

- **Balance Coverage vs. Maintenance:** Keep E2E tests focused on a few representative workflows; rely on unit/integration tests for logic. Too many E2Es can cause brittleness and slow CI.
- **Inject Test Harnesses:** Provide helper classes for testing HTTP endpoints with different auth states, including multi-tenant scenarios. This will reduce boilerplate and make adding tests cheaper.
- **Parameterize Scenarios:** Use theories or parameterized tests to cover multiple edge cases with less code, especially for board operations (moving cards to first/last positions, empty columns, etc.).
- **Monitor Test Flakiness:** Record and track any intermittent failures; configure CI to retry flaky tests but also investigate underlying causes.

## 4. How to Use This Checklist

1. **Review & Prioritize:** Scan the ideas and flag which ones align with your product vision and resources. Use the existing horizon structure (A/B/C) to place new tasks into an appropriate timeframe (e.g. Horizon D for long-term explorations).
2. **Create Issues:** For selected items, create new GitHub issues using the established naming convention (e.g. COL-01 for collaboration features, INT-01 for integrations). Define acceptance criteria, dependencies and suggested branches.
3. **Update Masterplan:** Incorporate these items into IMPLEMENTATION\_MASTERPLAN.md and update STATUS.md when goals are achieved. Document any changes in testing strategy in TESTING\_GUIDE.md.
4. **Iterate & Refine:** Periodically revisit this checklist; mark completed items, reprioritize outstanding tasks, and add new ideas gleaned from user feedback or further research.

By broadening Taskdeck's horizon beyond the current roadmap while continuing your disciplined engineering practices, you can evolve the project into a professional, extensible platform that supports small teams and scales to larger organizations.

- 
- 1 SEC-01: Enforce auth on legacy controllers  
<https://github.com/Chris0Jeky/Taskdeck/issues/58>
  - 2 SEC-04: Standardize API error contract assertions  
<https://github.com/Chris0Jeky/Taskdeck/issues/44>
  - 3 4 IMPLEMENTATION\_MASTERPLAN.md  
[https://github.com/Chris0Jeky/Taskdeck/blob/main/docs/IMPLEMENTATION\\_MASTERPLAN.md](https://github.com/Chris0Jeky/Taskdeck/blob/main/docs/IMPLEMENTATION_MASTERPLAN.md)
  - 5 UX-02: Drag/edit interaction safety guardrails  
<https://github.com/Chris0Jeky/Taskdeck/issues/45>
  - 6 PACK-05: Deterministic QA/E2E fixture packs from manifests  
<https://github.com/Chris0Jeky/Taskdeck/issues/51>
  - 7 DEBT-02: Log query scalability pass  
<https://github.com/Chris0Jeky/Taskdeck/issues/53>
  - 8 9 10 TESTING\_GUIDE.md  
[https://github.com/Chris0Jeky/Taskdeck/blob/main/docs/TESTING\\_GUIDE.md](https://github.com/Chris0Jeky/Taskdeck/blob/main/docs/TESTING_GUIDE.md)